

Local Address Repair with a LoRA-Tuned MiniCPM5-1B: Rules Floor, Base, and SFT on a Frozen German Test Set

Eduard Beloiu
MbitAI

September 2026

Abstract

Dirty address records break migration and operations work, and a repair that silently changes a correct field is worse than one that asks for review. This report fine-tunes `openbmb/MiniCPM5-1B` with LoRA supervised fine-tuning to repair messy German named-address records on a local machine, and compares three systems on the same 2,000 clean-gold-grouped held-out records: a deterministic rules floor, the untuned base model, and the v3 tuned model. Every output must satisfy a fixed JSON contract with a field-level change list and a review route, and every record writes one append-only audit entry. The v3 model reaches repair precision 0.6984, recall 0.1991, and F1 0.3099, versus 0.1860 for the rules floor and 0.0192 for the base model. It makes one empty-field fill against 1,039 for the base model, while review precision is 0.9992. Two exhibits show both sides of that trade: a name-field over-edit and a correct refusal to invent a postcode. Training, export, and evaluation use pinned revisions, dataset hashes, and committed result artifacts. The work evaluates public address records, not private customer data.

1 Introduction

Address data arrives messy: inconsistent spacing and casing, Unicode variants, punctuation, documented abbreviations such as `Muste:rstr.`, spelling defects, and values in the wrong fields. Deterministic normalization covers the mechanical cases. The open question is whether a small local model can cover the rest without inventing information that was never in the input.

This report answers that question narrowly. The system reads one dirty German named-address record and returns a normalized record, a list of field-level changes, and the fields a human should review. It does not geocode, match entities, or fill in missing address data. A missing road, postcode, or house number must be flagged for review rather than guessed. That rule shapes the whole design: the scorer counts filling an empty field as invention even when the guess matches gold, and it counts a correct refusal as a recall miss. The final numbers must be read with that rule in mind (§6).

This technical report makes three contributions:

- an output contract (JSON Schema plus semantic checks) with a per-record audit log and a review route, shared by all three systems;
- a LoRA fine-tune of MiniCPM5-1B on evidence-preserving targets with checkpoint selection on honesty metrics (review precision, damage, F1), never recall alone; and
- a frozen three-way evaluation on 2,000 held-out records with two failure exhibits that show where the tuned model over-edits and where it correctly refuses.

2 Related work

The data comes from Clean Me If You Can, which supplies paired dirty and clean address tables with an error taxonomy, a published download, and ODbL terms [1, 2]. We use only its German named-address slice. The project follows the same evidence standard as our earlier log-parsing work: pinned inputs, a deterministic floor, a per-decision audit trail, and failure exhibits in the report [4].

MiniCPM5-1B is a 1B-parameter model published by OpenBMB [5, 6]. OpenBMB publishes a TRL plus PEFT recipe for its LoRA adapters with assistant-only loss [7], which this project follows. LoRA [8], TRL [9], and PEFT [10] are standard tooling; what this report adds is the honesty-gated selection and the frozen comparison. Both base and tuned checkpoints run locally through the same `llama.cpp` build [11] as quantized GGUF files, so no address data leaves the machine.

3 Task and output contract

The input is one dirty record with six fields: `name`, `road`, `house_number`, `postcode`, `locality`, and `country_code`. The output must parse against a fixed JSON Schema:

```
{
  "clean_record": {
    "name": "Example GmbH",
    "road": "Musterstraße",
    "house_number": "12",
    "postcode": "80331",
    "locality": "Muenchen",
    "country_code": "DE"
  },
  "changes": [
    {
      "field": "road",
      "from": "Musterstr.",
      "to": "Musterstraße"
    }
  ],
  "needs_review": []
}
```

Semantic validation then checks that every change agrees with the dirty input and the returned record, that no field is both changed and reviewed, and that every changed field has an audit entry. The runtime writes one append-only JSONL audit record per input with the input fingerprint, model and prompt revision, parsed output, changed fields, schema and semantic results, `needs_review`, latency, and the score when gold data is available.

4 Data

Address data come from the AddressTable release (Zenodo record 20841898, version v1.0 dated 2026-06-26, DOI [10.5281/zenodo.20841898](https://doi.org/10.5281/zenodo.20841898), ODbL-1.0 [2, 3]). The full archive (`AddressTable.zip`, 7,263,043,993 bytes) was downloaded and checksum-verified; we use only the German named slice (374,533 aligned dirty/clean pairs). Entity identifiers were split before any training input was built: 5,000 training entities, 1,000 validation entities, and 2,000 held-out test entities, with no entity or clean target crossing splits. The training set may use controlled corruptions generated from the clean training partition; the test set uses only the source dataset’s original paired rows. Raw records are never committed; manifests record archive, per-file, and split hashes.

5 Method

All three systems share the contract from §3, the prompt (v2), the decoder settings (temperature 0.0, `top_p` 1.0, seed 7, 512 max tokens), the `llama.cpp` revision, and the Q4_K_M quantization.

Rules floor. Deterministic normalization for whitespace, casing, Unicode, documented abbreviations, and format checks. It never touches a field it cannot justify, so its precision is high and its recall is low by construction.

Base. Untuned `openbmb/MiniCPM5-1B` (revision `87179e5`) with the production prompt, served as `Q4_K_M GGUF` through a local `llama-server` with reasoning disabled (`n_ctx 2048`).

SFT. LoRA rank 16 (`alpha 32`, dropout 0.05) on the official target modules, assistant-only loss, 3 epochs over 5,000 evidence-preserving targets (about 939 steps, learning rate `2e-4`, cosine schedule, batch 2 with 8 accumulation steps). The definitive run trained on a Colab T4 in fp16 for 4,574.6 seconds; validation scoring took 2,364.9 seconds. Checkpoints were selected on validation data only with an honesty score, the mean of review precision, 1-minus-damage, and F1. The winner, `checkpoint-800`, scored 0.7708 (review precision 1.0, damage 0.0209, F1 0.3333 on a deterministic 100-record validation subsample). The adapter (`7103451b...d77ec712`) was merged into the base model and converted to `Q4_K_M GGUF` with pinned `llama.cpp` revision `b31b71f`. The training chat template differs from the inference template by design, following the official MiniCPM recipe; both are pinned and the difference is recorded, not unified.

6 Evaluation

The scorer compares each predicted record against its paired gold record field by field, with exact match after Unicode normalization and stripping. A correct repair requires evidence: the dirty field must be non-empty, so filling an empty field is invention even when the guess matches gold, and it never counts as a true positive. A correct abstention (empty field kept empty and flagged) still counts as a recall miss. This is deliberate: the test set and scorer stay unchanged on abstention versus recall, and this section owns the tension instead. Each result is one deterministic run on the fixed 2,000-record input, so variance estimates and significance tests do not apply.

Table 1: Frozen test results, $n = 2,000$. Contract is schema AND semantics (usable-output rate). Latency is per-record model time.

System	Prec	Rec	F1	Damage	Invent	Rev P	Schema	Contract	p50
Rules	0.9369	0.1033	0.1860	0.0	0	1.0	1.0	1.0	0 ms
Base	0.0281	0.0146	0.0192	0.0235	1039	0.5445	0.7175	0.0	2237.1 ms
SFT v3	0.6984	0.1991	0.3099	0.0237	1	0.9992	0.999	0.9535	1443.7 ms

The v3 tuned model improves repair F1 over both the rules floor (0.3099 vs. 0.1860) and the base model (0.0192), with higher repair precision than the base (0.6984 vs. 0.0281). It makes one empty-field fill against 1,039 for the base, whose contract validity is 0.0. It also makes 88 unsupported additions, where a prediction extends a non-empty input value with extra tokens. Damage is 0.0237, slightly above rules (0.0) and base (0.0235). Review precision 0.9992 with record coverage 0.809 says the review route works: when the model asks for a human, it is almost always right to ask.

By field, the v3 model is strongest where the evidence is mechanical: road (precision 0.9233, recall 0.6066) and house number (0.9322, 0.1378), followed by locality (0.8700, 0.3841). The name field carries most of the damage (rate 0.0754, 94 damaged clean fields; precision 0.3724, recall

0.1182). Postcode recall is 0.0066 and country code recall is 0.0, not because the model regularly invents them but because it abstains: country-code review has precision 1.0. Correct abstentions lower the reported recall. Read recall as fixed without guessing, not gaps closed.

Cost on an 8 GB Mac (Metal, shared local runtime): rules take seconds; base took about 79.5 minutes for 2,000 records (p50 2237.1 ms, p95 3969.4 ms); v3 SFT took about 50.8 minutes (p50 1443.7 ms, p95 2293.3 ms). A startup probe measured model load from server launch to `/health`: 1561.8 ms for base and 661.7 ms for v3 SFT. Peak resident memory during the one-second post-load probe was 810.6 MB and 816.8 MB respectively; the rules evaluator used 31.4 MB. These are startup-memory measurements, separate from the per-record latency figures.

7 Failure exhibits

Exhibit 1: name damage. The dirty and gold names are both `Helmholtz-Gymnasium`, with locality `Karlsruhe`. The v3 model returns `Helmholtz-Gymnasium Karlsruhe`, copying the locality into the name. Road `Kaiserallee`, house `6`, postcode `76133`, and locality are preserved; only the missing `country_code` is reviewed. The output passes schema and semantic validation, but the name change is an unsupported addition and a wrong repair. This is representative of the v3 name-field weakness: the name field has 0.0754 damage and 47 unsupported additions.

Exhibit 2: correct refusal. The dirty record reads `Schloss Wackerbarth`, road `Wackerbarthstr.`, house `1`, empty postcode, empty locality, and empty country code. The v3 model repairs the road to `Wackerbarthstraße`, leaves the postcode empty, and flags the postcode and country code in `needs_review`. The gold postcode (`01445`) and locality (`Radebeul`) were not visible at run time; guessing either would have been invention. The model fixes what the input supports and stops where the evidence stops. The scorer counts the kept-empty postcode as a recall miss by design.

8 Limitations

This work evaluates public address records, not private customer data, and supports no production safety claim. The tuned model still damages clean name fields at a rate of 0.0754 and makes 47 unsupported name additions; any deployment would need a name-field gate or mandatory review for name edits. Correct abstentions depress recall by construction, so the headline recall understates how often the model does the right thing and overstates nothing. Runtime was measured on one 8 GB machine; the memory figures are startup probes and do not claim a full-run process maximum. The frozen comparison is one deterministic run per system. The validation-subsample scores that picked the checkpoint used the HF generate path, while the frozen scores use the GGUF server path; both are pinned and reported, and the frozen numbers govern. No prompt, normalizer, model, or test-data change followed the frozen result.

9 Reproduction

The repository is `TMFNK/local-slm-de-address-repair` [12] at evaluation commit `bd50ecc`; the v3 Colab training run used commit `3464395`. Setup uses Python 3.12 with a `uv` lockfile. Data

preparation, smoke baseline, Colab training, frozen evaluation, and the report entry point each have one command (`docs/REPRO.md`); `make_report.py` remains a placeholder and is not treated as a result. Each frozen run checks the manifest hash before scoring and writes ignored per-record `audit.jsonl` plus committed per-system `metrics.json`; the shared `freeze.json` reflects the last run, so comparisons must use the per-system files.

Table 2: Pinned revisions and hashes. Full values live in `configs/` and the committed `metrics.json` files.

Artifact	Pin
Base model	<code>openbmb/MiniCPM5-1B</code> rev <code>87179e5</code>
Base GGUF	<code>MiniCPM5-1B-Q4_K_M.gguf</code> rev <code>3d55fac</code>
SFT adapter	<code>7103451b...d77ec712</code> (checkpoint-800)
<code>llama.cpp</code>	<code>b31b71f</code>
Prompt / decode	<code>v2</code> ; temp 0.0, top_p 1.0, seed 7
Dataset	Zenodo 20841898 (ODbL-1.0); dirty <code>2ef8ab14</code> , clean <code>78c852e3</code>
Test manifest	records <code>160eec15</code> (2,000 entities)

References

- [1] D2IP-TU Berlin, “Clean Me If You Can,” GitHub (paired dirty/clean address tables, error taxonomy, ODbL data terms). <https://github.com/D2IP-TUB/Clean-Me-If-You-Can>
- [2] “AddressTable v1.0,” Zenodo record 20841898, 2026-06-26. doi:10.5281/zenodo.20841898
- [3] Open Data Commons, “Open Database License (ODbL) v1.0.” <https://opendatacommons.org/licenses/odbl/>
- [4] E. Beloiu, “Trail: A Deterministic-First Log Template Miner with a Per-Decision Audit Trail.” Technical report, MbitAI, September 2026.
- [5] OpenBMB, “MiniCPM5-1B,” Hugging Face (Apache-2.0). <https://huggingface.co/openbmb/MiniCPM5-1B>
- [6] Y. Hu et al., “MiniCPM: Unveiling the potential of small language models with scalable training strategies.” 2024. <https://arxiv.org/abs/2404.06395>
- [7] OpenBMB, “MiniCPM finetune with TRL and PEFT,” <https://github.com/OpenBMB/MiniCPM/blob/main/docs/finetune/trl.md>
- [8] E. J. Hu et al., “LoRA: Low-rank adaptation of large language models.” 2021. <https://arxiv.org/abs/2106.09685>
- [9] L. von Werra et al., “TRL: Transformer Reinforcement Learning,” GitHub. <https://github.com/huggingface/trl>
- [10] S. Mangrulkar et al., “PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods,” GitHub. <https://github.com/huggingface/peft>
- [11] G. Gerganov et al., “llama.cpp,” GitHub (MIT). <https://github.com/ggml-org/llama.cpp>
- [12] E. Beloiu, “local-slm-de-address-repair” (Apache-2.0; data ODbL-1.0). <https://github.com/TMFNK/local-slm-de-address-repair>